Repository cleanup report

English version of ../CLEANUP_REPORT.md. Date: 26.09.2026. Repository Pavel1778/rtk-crm, branch dev, commit 232ad7a.

Outcome

No history cleanup is required. The check showed that there is no personal data, no secrets and no junk artefacts in the history, so rewriting history (git filter-repo/BFG) was not performed: it would have changed the hashes of 185 commits and forced every participant to re-clone, with no benefit whatsoever.

What was checked

Check	Command	Result
`.env`, `.pem`, `.key` ite in history	`git log --all --full-history -- '*.env*' '*.pem' '*.key' '*.db' '*.sqlite*'`	only `backend/.env.example`, `frontend/.env.example`, `infra/yandex-cloud/.env.prod.example` — templates with placeholders
Real secret values	`git log --all -p -- '*.env*' | grep -iE '(SECRET_KEY|PASSWORD|TOKEN|API_KEY)='`	matches only with empty values and commented-out lines
Private keys	searching `BEGIN ... PRIVATE KEY` across the whole history	0
Supabase/ GigaChat tokens	searching `eyJ...` and `service_role`	0
Files with the case holder's personal data	`git log --all --full-history -- "Загрузка пользователей.xlsx" "Данные оплат.json" "Вендоры.xlsx"`	absent from the history
Personal data in history	searching SNILS and passport patterns	only the synthetic placeholder `123-456-789 00` in tests
Junk in the index	`git ls-files | grep -iE 'node_modules|__pycache__|dist/|build/ |.venv|.idea|.DS_Store'`	empty
Large files	`git rev-list --objects --all | git cat-file --batch-check | sort`	at most 1.91 MB (`docs/presentation/template-lct2026.pptx`), the 50 MB threshold is never reached
Traces of third-party tools	`git log --all --format='%b'` for authorship trailers and generator signatures	empty
Third-party assistant configs	`git ls-files` for assistant configuration files in the root and subdirectories	empty
Absolute build-environment paths	searching `home/<build-user>` and preview domains	only in the ignored caches, removed; nothing in tracked files

A clarification about the previously deleted values

`docs/VERIFICATION_REPORT.md` (the item about old secrets) notes that previously published credentials cannot be considered revoked. The history check refines that assessment:

- **There are no real third-party credentials in the history.** A search across all 952 blob objects found neither real Supabase hosts (of the form `<project-ref>.supabase.co`), nor JWTs, nor GigaChat access keys. Only placeholders occur in the history: `<host>.pooler.supabase.com`, `<service>.onrender.com`, `<password>`, `<cluster-id>`.
- **What the repository did contain** were weak defaults for local runs, deleted in `9643426`:

Value	Where it was	Risk
`SECRET_KEY: change-me-32-chars-minimum`	`docker-compose.yml`	JWT forgery if the contour is brought up without an override
`SECRET_KEY: rtk-crm-2026-hackathon-secret-key-32chars`	`docs/DEPLOYMENT.md`	the same
`POSTGRES_PASSWORD: rtk_password`	`docker-compose.yml`	database access

All of them belong to the local contour (`postgres:5432`, `localhost`), not to cloud services.

What follows from this. Rotating third-party keys (Supabase, Render) is not required: they were never in the repository. But if a contour was deployed from `docker-compose.yml` without overriding `SECRET_KEY`, then `change-me-32-chars-minimum` reached a running service and must be replaced: a JWT signature on such a key is trivially forged. The current code prevents this — in production `Settings` rejects the default and too-short key, and Compose requires it explicitly (`SECRET_KEY: ${SECRET_KEY:?SECRET_KEY is required}`).

Synthetic data instead of personal data

The import sets are assembled in memory by tests (`tests/test_excel_import.py`, `tests/test_import_report.py`) and contain deliberately fictitious values. The `test_excel_import.py` test separately verifies that the import does not store SNILS, passport or address fields: the sensitive columns are dropped at the parsing stage. That is why separate anonymised copies in `docs/examples/` are unnecessary — their role is played by the fixtures and the sample reports (`docs/examples/README.md`).

What was done

1. `.gitignore` was extended with patterns for keys and certificates: `*.pem`, `*.key`, `credentials.json`, `service-account.json`, `service_account.json`, `key.json`, `*.p12`, `*.pfx`. Previously these files were not listed explicitly, although they could not have reached the repository — they were simply never created. The patterns were verified with `git check-ignore`: tracked files are unaffected.
2. `docs/examples/README.md` was added, explaining where the data comes from.

Repository state

Indicator	Value
Commits	185
Tracked files	276
`.git` size	19 MB
Working files size	15 MB
Largest file in history	1.91 MB

Conclusion

There is no risk of personal data leaking through the repository history. An additional history rewrite is unnecessary and would be harmful: it breaks commit signatures, requires a force-push to `main` and forces every participant to re-clone the repository.